

Calidad de software: estándares, modelos y metodologías

Breve descripción:

Este componente desarrolla competencias para asegurar la calidad del software conforme a estándares y buenas prácticas de la industria. Aborda fundamentos, modelos y normas como ISO/IEC, IEEE y CMMI. Integra enfoques tradicionales y ágiles, métricas, documentación técnica y herramientas de apoyo para planificar, verificar y mejorar procesos en proyectos.

Mayo 2026

Tabla de contenido

Introducción	4
1. Fundamentos de calidad del software	7
1.1. Conceptos básicos de calidad	7
1.2. Gestión de procesos	12
2. Estándares de calidad para el desarrollo de software	19
2.1. Familia ISO/IEC 25000 (SQuaRE).....	19
2.2. Modelo ISO/IEC 15504	20
2.3. Estándares IEEE.....	22
3. Modelos de calidad de software	33
3.1. CMMI (Capability Maturity Model Integration)	33
4. Metodologías de desarrollo de software	47
4.1. Metodologías tradicionales	47
4.2. Metodologías ágiles.....	50
4.3. Calidad en entornos ágiles vs tradicionales	53
5. Disciplinas de calidad de software	56
5.1. Personal Software Process (PSP)	56
5.2. Team Software Process (TSP).....	58
6. Administración del proceso personal de construcción de software	62

6.1.	Fundamentos y principios.....	62
6.2.	Niveles del PSP (PSP0 a PSP3).....	63
6.3.	Instrumentos de control	64
6.4.	Método PROBE	67
6.5.	Estadísticas aplicadas.....	69
6.6.	Herramientas informáticas de apoyo	71
7.	Documentación en procesos de calidad	73
7.1.	Estándares de documentación	73
7.2.	Formatos y plantillas.....	75
7.3.	Gestión documental	77
	Síntesis	79
	Glosario	80
	Referencias bibliográficas	81
	Créditos	82

Introducción

En un entorno donde las organizaciones dependen cada vez más de soluciones tecnológicas para operar y competir, la calidad del software se consolida como un factor estratégico. Los errores en los sistemas pueden generar pérdidas económicas, afectar la reputación institucional y comprometer la seguridad de la información. Por ello, la gestión adecuada de las pruebas y el aseguramiento de la calidad resultan componentes esenciales a lo largo del ciclo de vida del desarrollo de software.

Este componente proporciona los fundamentos conceptuales, normativos y prácticos necesarios para planificar, ejecutar, controlar y mejorar los procesos de verificación y validación, en coherencia con estándares internacionales y buenas prácticas de la industria. Mediante una modalidad virtual y un enfoque aplicado, se desarrollan competencias orientadas a garantizar productos de software confiables, eficientes y alineados con los requisitos establecidos, contribuyendo a la mejora continua de los procesos tecnológicos en las organizaciones.

Partiendo de lo anterior, se invita a que acceda al siguiente video, el cual relaciona la temática a tratar durante este componente formativo:

Video 1. Calidad de software: estándares, modelos y metodologías



[Enlace de reproducción del video](#)

Transcripción del video: calidad de software: estándares, modelos y metodologías

En el desarrollo de software, la calidad se concibe como un eje transversal que impacta los procesos, los productos y la satisfacción del usuario. Es así como este componente formativo inicia con los fundamentos de la calidad del software, abordando conceptos clave como la calidad del producto y del proceso, el aseguramiento y control de calidad, la verificación y validación, así como los costos de la calidad, integrados a la gestión por procesos, el ciclo PHVA y la mejora continua.

Posteriormente, se analizan los estándares internacionales de calidad, que proporcionan criterios formales para evaluar el software y los procesos de desarrollo, así como los modelos de calidad que permiten comprender los niveles de madurez y su aporte a la mejora organizacional. Este marco se complementa con el estudio de las metodologías de desarrollo, tanto tradicionales como ágiles,

destacando cómo se gestiona la calidad en contextos predictivos y adaptativos, y los retos asociados a cada enfoque.

Finalmente, el contenido integra las disciplinas de calidad personal y de equipo, orientadas a la disciplina, la medición y la toma de decisiones basada en datos, concluyendo con la documentación y gestión documental como soporte esencial para la trazabilidad, el control de versiones y la auditoría, consolidando una visión integral de la calidad del software enfocada en la mejora continua y el desarrollo de productos confiables.

1. Fundamentos de calidad del software

Este apartado presenta una visión general de los fundamentos de la calidad del software, entendida como un elemento estratégico para garantizar soluciones tecnológicas confiables, eficientes y alineadas con las necesidades de los usuarios y de las organizaciones. Se abordan los principios que sustentan la prevención de errores, la correcta gestión del trabajo y la mejora progresiva de los resultados, promoviendo una cultura orientada a la calidad, la evaluación sistemática y el fortalecimiento continuo de los procesos de desarrollo.

1.1. Conceptos básicos de calidad

A través del siguiente pódcast, se explicarán los conceptos básicos de calidad en el contexto del desarrollo de software, fundamentales para comprender cómo se evalúa y asegura que un producto cumpla con las expectativas del usuario y de la organización. En este espacio reflexivo, se abordará la diferencia entre calidad del producto y calidad del proceso:

Transcripción del Podcast. Calidad del producto vs. calidad del proceso
Mujer: según estudios de la industria, corregir un defecto en producción cuesta hasta cien veces más que corregirlo durante el diseño. Cien veces. Ese número no habla de código ni de bugs ni de versiones. Habla del proceso, de lo que ocurre cuando la calidad se revisa al final en lugar de construirse desde el inicio. Hombre: en el desarrollo de software hay que distinguir dos dimensiones de la calidad que van siempre de la mano pero que responden a planteamientos

diferentes. La primera es la calidad del producto: todo lo que el usuario experimenta cuando usa el software.

Mujer: ¿hace lo que debe hacer con precisión? ¿Responde rápido bajo carga real? ¿Es seguro? ¿Se puede usar sin necesitar un manual? ¿Aguanta entradas inválidas sin colapsar? Esas preguntas tienen respuestas medibles. La tasa de defectos que llegan a producción, su severidad, los tiempos de respuesta, la satisfacción de los usuarios, las incidencias que reportan. Cada indicador cuenta algo sobre lo que se construyó.

Hombre: y hay indicadores que van más allá de lo funcional. El cumplimiento de requisitos no funcionales como el rendimiento, la seguridad o la mantenibilidad. La robustez del sistema ante condiciones inesperadas o la aceptación del usuario en pruebas reales. Todo eso forma parte de lo que se mide cuando se evalúa la calidad del producto.

Mujer: la calidad del proceso cuestiona si ese equipo puede producir buenos resultados de forma consistente. La única manera de responder esa pregunta es con prácticas.

Hombre: prácticas como revisar el código antes de integrarlo, gestionar los defectos con trazabilidad, y cuando algo sale mal, sentarse a entender la causa raíz para que no vuelva a ocurrir. Eso es lo que hace un proceso maduro: aprender de cada ciclo y mejorar con el tiempo.

Mujer: cuando los requisitos están claros antes de arrancar, cuando el diseño se revisa antes de implementarse, cuando el control de cambios funciona y la

integración continua detecta los problemas a tiempo, el proceso está trabajando a favor del equipo. Y eso se nota en los resultados.

Hombre: un proceso maduro detecta los problemas cuando son baratos de resolver: en el diseño, en la revisión de código, en las pruebas planificadas. Cuando el proceso falla en esos momentos, los problemas llegan a producción donde el costo de corregirlos se multiplica.

Mujer: la calidad no llega sola al final del desarrollo. Se construye en cada decisión que se toma desde el principio.

Hombre: quien aprende a trabajar con las dos dimensiones en mente puede leer un proyecto desde adentro: saber si los indicadores del producto están bien y si el proceso que los produjo es sostenible. Esa capacidad de lectura es lo que define a un verdadero profesional.

Mujer: gracias por su compañía. Nos escuchamos en el próximo programa.

- **Aseguramiento de calidad (QA) vs control de calidad (QC)**

Aunque se usan como sinónimos, en gestión profesional son complementarios y distintos. Estas son sus especificaciones:

- **QA – Quality Assurance (aseguramiento de calidad)**

Enfoque: preventivo, orientado al proceso.

Propósito: evitar que los defectos nazcan, haciendo que el proceso sea capaz de producir calidad.

Ejemplos de actividades QA (en software/testing):

- Definir política de calidad y estrategia de pruebas.
- Establecer estándares (nomenclatura, criterios de aceptación, definición de “done”, guías de documentación).
- Auditorías internas de cumplimiento (¿se hace trazabilidad?, ¿se registran defectos correctamente?).
- Revisión de procesos (postmortem, lecciones aprendidas, mejora del flujo de defectos).
- Gestión de métricas y tableros.
- Asegurar que existan criterios de entrada/salida en pruebas (gates de calidad).

○ **QC – Quality Control (control de calidad)**

Enfoque: detectivo/correctivo, orientado al producto.

Propósito: encontrar defectos en el software antes de que llegue a producción.

Ejemplos de actividades QC:

- Ejecución de pruebas (manuales/automatizadas): unitarias, integración, sistema, aceptación.
- Pruebas no funcionales: rendimiento, seguridad, compatibilidad, usabilidad.
- Inspecciones del producto: revisión de interfaz, validación de requisitos implementados.

- Reporte de defectos y verificación de correcciones (retesting y regresión).

- **Verificación y Validación (V&V)**

Estas prácticas son centrales en cualquier enfoque de calidad, ya que permiten evaluar el software tanto desde el cumplimiento técnico como desde su adecuación al uso real, asegurando coherencia entre lo que se construye, lo que se define y lo que el usuario necesita. A partir de ello, cumplen las siguientes acciones:

- **Verificación**

Idea: “¿Se construye el producto correctamente?”

Se centra en conformidad con especificaciones y diseño. Evalúa consistencia interna con lo planeado.

Objetivo: detectar desviaciones técnicas temprano.

Producto típico: evidencias de revisión, checklist, hallazgos, métricas de cobertura, resultados de análisis.

- **Validación**

Idea: “¿Se construye el producto correcto?”

Se centra en adecuación al uso real y necesidades del usuario/negocio.

Objetivo: asegurar que el software entregue valor en contexto.

Producto típico: actas de aceptación, evidencias de cumplimiento de historias, resultados de pruebas con usuarios.

- **Costos de la calidad**

Los costos de calidad son la forma gerencial de demostrar que invertir en calidad es rentable. Tradicionalmente se agrupan en cuatro categorías:

- **Costos de prevención.** Inversión para evitar defectos.
- **Costos de evaluación (o de detección).** Costos para encontrar defectos antes de liberar.
- **Costos de fallos internos.** Defectos detectados antes de producción (pero ya ocurrieron).
- **Costos de fallos externos.** Defectos detectados en producción (los más caros).

Principio clave: el costo de un defecto crece dramáticamente cuanto más tarde se detecta. Por eso la gestión de pruebas y QA busca mover detección y prevención “a la izquierda” (shift-left).

1.2. Gestión de procesos

La gestión de procesos permite organizar y controlar el trabajo de forma sistemática, asegurando que las actividades se ejecuten de manera coherente, medible y orientada a resultados.

- **Enfoque basado en procesos**

Gestionar calidad en software implica tratar el desarrollo y las pruebas como un sistema de procesos interrelacionados, no como actividades aisladas. Un proceso se

describe por: entradas (requisitos, historias, diseños, código), actividades (diseñar pruebas, ejecutar, registrar defectos), salidas (evidencias, métricas, software probado), responsables (roles claros), recursos (herramientas, ambientes, datos), así como reglas y estándares (definiciones, plantillas, criterios).

Los siguientes son los beneficios del enfoque por procesos en pruebas/QA:

- **Repetibilidad.** El resultado no depende “del héroe” sino del sistema.
- **Trazabilidad.** Se puede explicar y auditar lo que se hizo.
- **Control.** Se detectan cuellos de botella (ej. ambientes, datos, aprobaciones).
- **Mejora.** Se identifica qué cambiar para reducir defectos y retrabajo.

Aplicación típica en testing:

- Proceso de planificación de pruebas (estrategia → plan → estimación).
- Proceso de diseño de pruebas (requisitos → casos → datos).
- Proceso de ejecución (build estable → ejecución → evidencias).
- Proceso de gestión de defectos (registro → triage → corrección → verificación).
- Proceso de cierre (métricas → lecciones aprendidas → mejora).

- **Ciclo PHVA (PDCA)**

El PHVA es un marco universal de mejora continua que en pruebas/QA que se traduce de forma muy concreta, así:

Planear

- Definir política/objetivos de calidad.
- Establecer estrategia y plan de pruebas.
- Definir criterios de entrada/salida (Definition of Ready/Done, gates).
- Planificar métricas (qué medir y cómo).
- Identificar riesgos de calidad (módulos críticos, cambios mayores, deuda técnica).

Hacer

- Ejecutar el proceso: diseño y ejecución de pruebas, automatización, revisión.
- Gestionar defectos con disciplina (severidad, prioridad, trazabilidad).
- Mantener evidencias y documentación (sin burocracia innecesaria, pero con rigor).

Verificar

- Analizar resultados: defectos por módulo, por tipo, tendencias.
- Verificar cumplimiento de criterios de salida.
- Auditar artefactos clave (plan, casos, reporte, trazabilidad).
- Evaluar efectividad del proceso: ¿qué tanto se detecta antes de producción?

Actuar

- Implementar mejoras: ajustes al proceso, automatización, estándares, capacitación.

- Eliminar causas raíz (no solo “arreglar el bug”).
- Actualizar plantillas, checklists, definición de “done”.
- Repriorizar riesgos y reforzar controles donde más impacta.

PHVA en modalidad virtual: muy útil porque obliga a formalizar acuerdos (criterios, trazabilidad, evidencias) y a usar métricas para coordinación remota.

- **Gestión por indicadores**

La gestión por indicadores evita decisiones “a ojo” y habilita control real. En pruebas/QA, las métricas deben cumplir tres condiciones:

- A.** Ser interpretables (que guíen acción).
- B.** Ser consistentes (misma definición siempre).
- C.** No incentivar trampas (ej. “cerrar por cerrar” defectos).

A continuación, se presentan indicadores clave con propósito gerencial, que permiten evaluar el estado del producto, la efectividad de las pruebas y la estabilidad de los procesos, facilitando la identificación de riesgos y la toma de decisiones oportunas en proyectos de software:

Indicadores de defectos

- Densidad de defectos: defectos por tamaño (KLOC, puntos de función, historias). Sirve para comparar módulos y detectar zonas de riesgo.
- Severidad de defectos: proporción críticos/altos vs medios/bajos. Sirve para medir riesgo real, no solo cantidad.

- Defectos escapados (leakage): defectos hallados en producción vs totales. Mide efectividad del proceso de pruebas.

Indicadores de efectividad de pruebas

- DRE (Defect Removal Efficiency): qué porcentaje de defectos se elimina antes de liberar. Refleja madurez de QA/QC y efectividad del pipeline.
- Cobertura de pruebas: por requisitos, por código, por riesgo. Debe interpretarse con cuidado: cobertura alta no garantiza calidad si los casos son pobres.

Indicadores de estabilidad y flujo

- Tasa de reapertura: defectos reabiertos / defectos cerrados. Indica baja calidad de correcciones o mala definición del defecto.
- Tiempo medio de ciclo del defecto: desde reporte hasta cierre verificado. Mide eficiencia operativa entre QA y desarrollo.
- Tendencia de defectos por sprint/release: si suben, se estancan o bajan. Permite decisiones: congelar alcance, aumentar pruebas, reforzar revisiones.

Indicadores de servicio/calidad del servicio de software

- Incidentes post-release (por semana/mes).
- Cumplimiento de SLA (si aplica).
- MTTR (tiempo medio de recuperación).

Estos indicadores conectan testing con el “servicio de software”, que es clave en tu competencia del curso.

Regla de oro: un indicador sin decisión asociada es ruido. Cada métrica debe responder:

- ¿Qué decisión habilita?
- ¿Qué acción se toma si empeora?

- **Mejora continua**

La mejora continua en calidad de software es un enfoque sistemático orientado a incrementar, de manera sostenida, la capacidad del proceso para prevenir defectos, detectar fallas tempranamente y entregar versiones estables con menor variabilidad. No se limita a “arreglar lo que salió mal” en una liberación; implica aprender del comportamiento del proceso y del producto para modificar prácticas, estándares, herramientas y criterios de control. En contextos de pruebas de software, la mejora continua conecta directamente con la gestión del servicio de software, porque reduce incidentes en producción, estabiliza el desempeño y mejora el cumplimiento de acuerdos de nivel de servicio (SLA) cuando existan.

En términos operativos, la mejora continua se apoya en tres pilares: (1) medición y evidencia, (2) análisis causal y (3) acciones de mejora verificables.

En síntesis, la mejora continua convierte la calidad en una capacidad organizacional acumulativa. Al aplicar este enfoque en pruebas de software, se fortalece el control del proceso (menos incertidumbre), se incrementa la confiabilidad del producto (menos fallos externos) y se establece una base para adoptar modelos de

madurez (como CMMI o ISO/IEC 15504) con evidencias cuantitativas que soporten decisiones de gestión.

2. Estándares de calidad para el desarrollo de software

La estandarización en ingeniería de software cumple una función estratégica: proporciona un lenguaje común, criterios objetivos de evaluación y marcos sistemáticos para garantizar calidad, repetibilidad y mejora continua.

2.1. Familia ISO/IEC 25000 (SQuaRE)

La familia ISO/IEC 25000, conocida como SQuaRE (Software Product Quality Requirements and Evaluation), reemplaza y amplía el modelo ISO/IEC 9126 y tiene como propósito establecer un marco integral para definir requisitos de calidad del producto software, medir y evaluar sus características, verificar el grado de cumplimiento y promover la integración de la calidad a lo largo de todo el ciclo de vida del software.

La estructura SQuaRE se organiza en cinco divisiones principales:

- ISO/IEC 2500n – Gestión de calidad
- ISO/IEC 2501n – Modelo de calidad
- ISO/IEC 2502n – Medición de calidad
- ISO/IEC 2503n – Requisitos de calidad
- ISO/IEC 2504n – Evaluación de calidad

Este enfoque integra modelo conceptual + métricas + evaluación sistemática.

2.2. Modelo ISO/IEC 15504

Conocido como SPICE (Software Process Improvement and Capability determination), es un estándar internacional y herramienta poderosa que proporciona un marco para evaluar y mejorar los procesos de desarrollo de software. Este modelo se centra en la capacidad de los procesos, lo que significa que no solo describe qué procesos deben existir, sino también cómo se pueden medir y mejorar para garantizar la calidad del software producido.

SPICE se basa en dos conceptos clave:

- Process Assessment

Se refiere a la evaluación de la madurez y capacidad de los procesos de software. Esto implica analizar cómo se llevan a cabo los procesos, identificar puntos fuertes y áreas que necesitan mejora.

- Capability Levels

SPICE define cinco niveles de capacidad para los procesos, que van desde el nivel 1 (inicial, donde los procesos son ad hoc y poco controlados) hasta el nivel 5 (optimizado, donde hay mejoras continuas y adaptaciones). Esto permite a las organizaciones identificar su posición actual y establecer un camino hacia la mejora.

La aplicación de SPICE en el desarrollo de pruebas de software implica varios pasos fundamentales:

- A. Definición de procesos de pruebas.** Primero, es importante definir claramente los procesos que se utilizarán en las pruebas. Esto incluye su planificación, el diseño de casos de prueba, la ejecución, el registro de

resultados y la gestión de defectos. SPICE ayuda a formalizar y documentar estos procesos.

- B. Evaluación de procesos.** Una vez definidos, la organización debe evaluar estas acciones utilizando el modelo de evaluación de SPICE. Esto implica recopilar datos sobre cómo se llevan a cabo los procesos de pruebas y compararlos con los niveles de capacidad establecidos en el modelo. Por ejemplo, se podría evaluar si las pruebas se planifican de manera sistemática o si se realizan de forma ad hoc.
- C. Identificación de mejoras.** Con base en la evaluación, se identifican las áreas que necesitan mejora. Esto puede incluir aspectos como la documentación deficiente de los casos de prueba, falta de seguimiento en la gestión de defectos o ineficiencias en la ejecución de pruebas automatizadas.
- D. Implementación de cambios.** Después de identificar las áreas de mejora, se implementan acciones correctivas y cambios en los procesos. Esto puede involucrar la capacitación del personal, la adopción de herramientas de gestión de pruebas o la creación de plantillas estandarizadas para casos de prueba.
- E. Monitoreo y re-evaluación.** Finalmente, SPICE promueve la idea de que la mejora es un esfuerzo continuo. Después de implementar cambios, es importante seguir monitoreando su efectividad en las actividades de prueba, recoger feedback y volver a evaluar con el modelo SPICE para verificar si se ha avanzado hacia niveles de capacidad más altos.

2.3. Estándares IEEE

El IEEE (Instituto de Ingenieros Eléctricos y Electrónicos) es la organización técnica profesional más grande del mundo. Sus estándares definen las buenas prácticas que guían el desarrollo y la verificación del software a nivel global.

A continuación, un poco de historia:

- **1987: IEEE 1008 - Establecido.** Se introduce una metodología sistemática para pruebas unitarias.
- **1993: IEEE 829 - Publicado.** Se establece el primer estándar para la documentación de pruebas.
- **1998-2008 - Revisiones y actualizaciones.** Los estándares se adaptan a metodologías ágiles y nuevos paradigmas.
- **2013: IEEE/ISO/IEC 29119 - Unificado.** Se crea una familia integral de normas para pruebas de software.

- **IEEE 829 — Documentación de pruebas**

El IEEE 829 es un estándar internacional desarrollado por el Institute of Electrical and Electronics Engineers (IEEE), que establece una estructura formal para la documentación del proceso de pruebas de software. Su propósito es garantizar que las actividades de prueba se planifiquen, ejecuten y documenten de manera sistemática, trazable y verificable, contribuyendo así a la calidad del producto y a la transparencia del proceso.

Este estándar no define cómo realizar técnicamente las pruebas, sino qué documentos deben existir y qué información mínima deben contener, promoviendo buenas prácticas de aseguramiento de calidad en proyectos de software.

El modelo de IEEE 829 contempla documentos principales, cada uno con un propósito específico dentro del ciclo de pruebas así:

- **Plan de pruebas**

El plan de pruebas es el documento estratégico que orienta y estructura todo el proceso de verificación y validación del sistema. En él, se define el alcance, especificando qué funcionalidades, módulos o requisitos serán evaluados y cuáles quedarán fuera del ciclo de pruebas.

Asimismo, establece el enfoque, determinando los tipos de pruebas que se ejecutarán, tales como pruebas unitarias, de integración, de sistema, de aceptación, de rendimiento o de seguridad, entre otras.

En cuanto a los recursos, se detallan el equipo humano responsable, las herramientas que se utilizarán y los ambientes de prueba requeridos para garantizar una ejecución adecuada.

Adicionalmente, el documento contempla el cronograma de actividades, los criterios de entrada y salida que regulan el inicio y cierre del proceso, así como los riesgos o factores potenciales que puedan afectar el desarrollo de las pruebas. De esta manera, el plan de pruebas se convierte en la guía formal que asegura organización, control y trazabilidad durante todo el ciclo de evaluación.

- **Especificación de diseño**

Traduce los requisitos del sistema en condiciones verificables. Define qué se va a probar y cómo se organizarán los casos de prueba.

Casos de prueba detallados: su objetivo es asegurar la cobertura adecuada de los requisitos y mantener trazabilidad entre lo solicitado y lo evaluado.

- **Especificación de procedimiento**

Este documento describe de manera detallada cada caso de prueba individual.

Incluye:

- a. Identificador único del caso.
- b. Objetivo específico.
- c. Condiciones previas.
- d. Datos de entrada.
- e. Procedimiento resumido.
- f. Resultado esperado.
- g. Criterios de aceptación.

Una vez definidos los documentos de planeación y especificación, el estándar IEEE 829 también contempla artefactos orientados al seguimiento, control y cierre del ciclo de pruebas. Estos documentos permiten registrar lo que realmente ocurrió durante la ejecución, gestionar los defectos detectados y consolidar los resultados finales, aportando evidencia objetiva para la evaluación de la calidad del software y la toma de decisiones, encontrando:

- **Registro de pruebas.** Es el documento que consigna la ejecución real de las pruebas realizadas. Incluye los casos ejecutados, la fecha y el responsable, el resultado obtenido (aprobado, fallido o bloqueado), las incidencias detectadas y las evidencias asociadas, como capturas o logs. Sirve como historial del proceso y facilita auditorías y revisiones posteriores.
- **Reporte de incidentes.** Describe los defectos o comportamientos inesperados encontrados durante la ejecución, generalmente debe incluir identificador del incidente, descripción detallada del problema, resultado esperado versus el obtenido, niveles de severidad y prioridad y el estado del incidente (abierto, en análisis, corregido, cerrado).
- **Reporte de resumen.** Se elabora al finalizar un ciclo de pruebas y consolida los resultados globales del proceso. En él se presenta el alcance cubierto, el total de casos ejecutados, el porcentaje de éxito y de fallos, los incidentes pendientes y la evaluación de los riesgos residuales. Además, incluye una recomendación sobre la liberación del sistema, con el fin de apoyar la toma de decisiones por parte de la gerencia o los stakeholders.

- **IEEE 730 — Aseguramiento de Calidad (SQAP)**

Es el mapa de ruta que permite a un equipo de software garantizar que la calidad no es un evento aislado al final del proyecto, sino un proceso continuo, integrado en cada fase del ciclo de vida. La siguiente tabla ejemplifica el proceso:

Tabla 1. Secciones del SQAP y su aporte al aseguramiento de la calidad

Sección del SQAP	Qué garantiza	Entregable principal
1. Propósito	Claridad sobre objetivos y alcance de la calidad.	Declaración de alcance del SQAP.
2. Referencias	Coherencia normativa y contractual.	Tabla de referencias citadas.
3. Gestión	Roles, responsabilidades y recursos claros.	Organigrama QA y matriz RACI.
4. Documentación	Trazabilidad y evidencia del proceso.	Lista de documentos del proyecto.
5. Estándares y prácticas	Uniformidad técnica en el equipo.	Guías de codificación y prácticas.
6. Revisiones e inspecciones	Detección temprana de defectos.	Calendario y actas de revisiones.
7. Gestión de riesgos	Anticipación de problemas de calidad.	Registro y matriz de riesgos.
8. Métricas de Calidad	Medición objetiva del nivel de calidad.	Dash board de indicadores de calidad.

- **IEEE 1008 — Pruebas unitarias**

Es el estándar internacional dedicado específicamente al proceso de Pruebas de Unidad (Unit Testing), cuyo objetivo es asegurar que los componentes mínimos del software funcionen correctamente de forma aislada antes de ser integrados al sistema completo. Se caracteriza por un enfoque sistemático y disciplinado que divide el trabajo en cuatro fases críticas —Planificación, Adquisición, Ejecución y

Verificación— permitiendo a los desarrolladores identificar errores en las etapas más tempranas. Cada fase es explicada a continuación:

- **Planificación - Definir el alcance**

En esta fase inicial, no se trata solo de "decir qué probar", sino de establecer las reglas del juego para cada unidad de código (una función, una clase o un módulo pequeño). Incluye:

- Identificación de la unidad: se determina qué componentes específicos serán sometidos a prueba.
- Definición de criterios de suficiencia: ¿Cuándo se considera que la unidad ha sido probada "lo suficiente"? (Ej. cobertura de código al 90 %).
- Evaluación de riesgos: identificar qué unidades son más críticas para el funcionamiento general del sistema.

- **Adquisición - Preparar casos y entorno**

Aquí se recolectan y crean los recursos necesarios. No se puede probar si no se tiene el "laboratorio" listo. Al respecto se tiene en cuenta:

- Diseño de casos de prueba: se definen las entradas (inputs), las condiciones de ejecución y, lo más importante, los resultados esperados.

- Preparación del entorno: configuración de herramientas de prueba, stubs (objetos simulados que reemplazan dependencias que aún no existen) y drivers (controladores que llaman a la unidad).
- Generación de datos: crear los datos de prueba, incluyendo casos de éxito y casos de error (datos inválidos).

- **Ejecución - Aplicar y registrar**

Es fase de “acción” donde se corren las pruebas diseñadas en la fase anterior sobre el código real y se tiene en cuenta:

- Corrida de pruebas: se ejecutan los casos de prueba en el entorno preparado.
- Registro de incidentes: cada vez que el resultado real no coincide con el esperado, se documenta como un fallo o anomalía.
- Logs de prueba: se genera una bitácora técnica que detalla qué se ejecutó, en qué orden y qué sucedió en el sistema.

- **Verificación - Confirmar criterios**

Es la fase de cierre donde se decide si la unidad "pasa" o "reprueba" y tiene en cuenta:

- Evaluación de resultados: se comparan los resultados registrados contra los criterios de suficiencia definidos en la fase de planificación.

- Análisis de errores: se determina si los fallos son errores de código, de diseño o incluso errores en el propio caso de prueba.
- Resumen de pruebas: se emite un reporte final de la unidad indicando si está lista para integrarse con el resto del sistema.

- **IEEE 1012 - Verificación y Validación (V&V) de software**

Es una de las guías más robustas y respetadas en la ingeniería de software, ya que define los procesos necesarios para asegurar que un sistema cumpla con sus requisitos y con las necesidades del usuario final.

¿Qué es Verificación y Validación (V&V)?

Aunque suelen usarse como sinónimos, la norma hace una distinción clara:

- Verificación. ¿Se está construyendo el producto correctamente? (Evaluar si el software cumple con las especificaciones de cada fase).
- Validación: ¿Se está construyendo el producto correcto? (Evaluar si el software final satisface las necesidades del cliente).

Los niveles de integridad (Integrity levels)

Lo que hace especial a la IEEE 1012 es que no es "talla única". La intensidad de las pruebas depende del riesgo. Define 4 niveles relacionados en la siguiente tabla:

Tabla 2. Niveles IEEE 1012

Nivel	Descripción	Consecuencia del fallo
4 (Alto)	Crítico	Pérdida de vidas humanas o desastre catastrófico.
3 (Medio)	Importante	Fallos en misiones críticas o grandes pérdidas económicas.
2 (Bajo)	Moderado	Inconvenientes operativos o fallos menores.
1 (Mínimo)	Informativo	Impacto insignificante.

A mayor nivel de integridad, la norma exige tareas de V&V más rigurosas y detalladas.

Estructura del Proceso de V&V

La norma organiza las actividades de V&V a lo largo de todo el ciclo de vida del software, no solo al final. Cubre procesos como:

- **Gestión.** Planificación de V&V, evaluación de riesgos y control de calidad.
- **Adquisición y suministro.** Revisión de contratos y planes de proveedores.
- **Desarrollo.** Incluye los requisitos, diseño, implementación (código) y pruebas.
- **Operación y mantenimiento.** V&V continuo durante el uso del software y actualizaciones.

Los beneficios de implementar IEEE 1012 son:

- **Detección temprana de errores.** Al verificar desde los requisitos, el costo de corregir errores es significativamente menor.
- **Mejora de la calidad.** Asegura que el producto final sea fiable y seguro.
- **Cumplimiento normativo.** Es esencial en sectores regulados como la medicina, aviación, energía nuclear y automotriz.

- **IEEE/ISO/IEC 29119 — Familia de estándares**

Esta familia define un marco integral para las pruebas de software, del cual se desprenden las siguientes partes:

Parte 1. Conceptos y definiciones.

Parte 2. Procesos de prueba.

Parte 3. Documentación de prueba.

Parte 4. Técnicas de prueba.

Parte 5. Pruebas en contextos ágiles (Keyword-Driven).

Uno de los errores más comunes es usar un estándar fuera de su contexto. Por ello, la esta tabla ayuda a elegir el correcto, según la situación del proyecto:

Tabla 3. Comparación de estándares de prueba

Estándar	IEEE 829	IEEE 730	IEEE 1008	IEEE 29119-2	IEEE 29119-4	IEEE 29119-5
Enfoque principal	Documentación formal del proceso de prueba.	Plan de aseguramiento de calidad.	Pruebas unitarias estructuradas.	Procesos completos de prueba.	Técnicas de diseño de prueba.	Pruebas en entornos ágiles.

Estándar	IEEE 829	IEEE 730	IEEE 1008	IEEE 29119-2	IEEE 29119-4	IEEE 29119-5
Aplica en	Todo el STLC.	Fase de planeación.	Desarrollo / Sprint.	Gestión de pruebas.	Diseño de casos.	Sprint / scrum.
Nivel de proyecto	Todos.	Grande.	Mediano.	Grande.	Todos.	Ágil.

3. Modelos de calidad de software

Son marcos de referencia que establecen criterios y características para evaluar y asegurar la calidad de un producto de software. Definen aspectos como funcionalidad, confiabilidad, usabilidad, eficiencia y mantenibilidad, permitiendo medir el cumplimiento de estándares y orientar la mejora continua del sistema.

3.1. CMMI (Capability Maturity Model Integration)

CMMI es un modelo de mejora de procesos que permite a las organizaciones evaluar y optimizar la manera en que desarrollan y gestionan sus proyectos. Su enfoque se basa en niveles de madurez que reflejan el grado de definición, control y mejora continua de los procesos organizacionales. Cada nivel representa una etapa evolutiva que debe cumplirse progresivamente, ya que el avance hacia niveles superiores requiere la consolidación de las prácticas establecidas en los niveles anteriores.

- **Niveles de madurez**

Un nivel de madurez es una plataforma evolutiva que describe el grado en que los procesos de una organización están definidos, gestionados, medidos y optimizados. Cada nivel representa un estado de madurez organizacional: para alcanzar un nivel superior se debe haber institucionalizado completamente todos los niveles anteriores. No es posible 'saltar' niveles. La siguiente tabla detalla estos niveles:

Tabla 4. Niveles de madurez

N	Nivel	Perfil	Descripción y características
1	Inicial (Initial)	Caótico	Procesos ad-hoc e impredecibles. Éxito dependiente de individuos. Proyectos frecuentemente sobre presupuesto y plazo. Alta variabilidad en resultados.
2	Gestionado (Managed)	Planificado	Los proyectos se planifican, monitorean y controlan. Los procesos se documentan y se adhieren. El éxito puede repetirse en proyectos similares.
3	Definido (Defined)	Estandarizado	Procesos bien definidos, adaptados desde el Proceso Estándar de la Organización (OSP). Toda la organización opera con prácticas coherentes.
4	Cuantitativamente gestionado	Medido	Se usan técnicas estadísticas (SPC) para comprender y controlar los subprocesos. Objetivos de calidad cuantitativos establecidos y monitoreados.
5	En optimización (Optimizing)	Innovador	Mejora continua e innovación. Se identifican y eliminan causas de variación. Alta capacidad de adaptación ante cambios de negocio y tecnología.

- **Áreas de proceso (Process Areas — PAs)**

Un Área de Proceso (PA) es un conjunto de prácticas relacionadas que, cuando se implementan colectivamente, satisfacen un conjunto de metas importantes para mejorar esa área. Cada PA tiene metas específicas (SG — Specific Goals) y metas genéricas (GG — Generic Goals), las cuales incluyen:

- **Metas específicas (SG).** Únicas de cada PA. Definen lo que debe lograrse en esa área particular.
- **Metas genéricas (GG).** Aplican a todas las PAs y describen el grado de institucionalización del proceso.

Las metas genéricas (GC) se dividen en:

- GG2. Gestionar el proceso definido.
- GG3. Definir el proceso.
- GG4. Gestionar cuantitativamente
- GG5. Optimizar.

A continuación, se presentan las áreas de proceso organizadas en cuatro categorías:

Tabla 5. Categoría 1. Gestión de proyectos (PM)

Sigla	Área de proceso	N° Nivel	Descripción y metas (SG)
PP	Planificación del proyecto	Nivel 2	Planes que definen actividades del proyecto. Estimación de tamaño, esfuerzo, costo y cronograma.
PMC	Monitoreo y control del proyecto	Nivel 2	Permite comprender el avance y tomar acciones correctivas oportunas cuando el proyecto se desvía del plan.
SAM	Gestión de acuerdos con proveedores	Nivel 2	Gestiona la adquisición de productos y servicios de proveedores externos mediante acuerdos formales. SG1: establecer acuerdos. SG2: satisfacer acuerdos.

Sigla	Área de proceso	N° Nivel	Descripción y metas (SG)
IPM	Gestión integrada del proyecto	Nivel 3	Gestiona el proyecto usando el proceso estándar de la organización adaptado al contexto.
RSKM	Gestión de riesgos	Nivel 3	Identifica y mitiga problemas potenciales antes de que ocurran, reduciendo su impacto adverso.
QPM	Gestión cuantitativa del proyecto	Nivel 4	Gestiona el proyecto con técnicas estadísticas para lograr objetivos cuantitativos de calidad y desempeño.

Tabla 6. Categoría 2. Ingeniería de procesos (PE)

Sigla	Área de proceso	N° Nivel	Descripción y metas (SG)
REQM	Gestión de requisitos	Nivel 2	Gestiona los requisitos del proyecto y asegura alineación entre planes, actividades y productos. SG1: gestionar requisitos.
RD	Desarrollo de requisitos	Nivel 3	Identifica, analiza y establece los requisitos del cliente, del producto y de sus componentes. SG1: req. Cliente. SG2: del producto. SG3: análisis.
TS	Solución técnica	Nivel 3	Diseña, desarrolla e implementa soluciones a los requisitos: arquitectura, componentes, documentación. SG1: seleccionar solución. SG2: diseño. SG3: implementar.
PI	Integración del producto	Nivel 3	Ensambla el producto a partir de sus componentes y asegura que el producto integrado funciona correctamente. SG1: preparar integración. SG2: compatibilidad. SG3: ensamblar.

Sigla	Área de proceso	N° Nivel	Descripción y metas (SG)
VER	Verificación	Nivel 3	Asegura que los productos de trabajo cumplen los requisitos especificados (¿construimos correctamente?). SG1: preparar. SG2: revisiones entre pares. SG3: verificar.
VAL	Validación	Nivel 3	Demuestra que el producto cumple su uso previsto en el entorno real del cliente (¿construimos lo correcto?). SG1: preparar validación. SG2: validar producto.

Tabla 7. Categoría 3. Soporte (S)

Sigla	Área de proceso	N° Nivel	Descripción y metas (SG)
CM	Gestión de configuración	Nivel 2	Establece y mantiene la integridad de los productos de trabajo, mediante identificación, control y auditorías. SG1: línea base. SG2: rastrear cambios. SG3: integridad.
PPQA	Aseguramiento de calidad de proceso y producto	Nivel 2	Provee visibilidad objetiva del proceso y de los productos de trabajo asociados. SG1: evaluar objetivamente. SG2: retroalimentación objetiva.
MA	Medición y análisis	Nivel 2	Desarrolla la capacidad de medición para apoyar las necesidades de información de gestión. SG1: alinear medición. SG2: proveer resultados.
DAR	Análisis de decisiones y resolución	Nivel 3	Analiza decisiones usando un proceso formal de evaluación basado en criterios establecidos. SG1: evaluar alternativas usando criterios formales.

Sigla	Área de proceso	Nº Nivel	Descripción y metas (SG)
CAR	Análisis causal y resolución	Nivel 5	Identifica causas de defectos y otros problemas, y actúa para prevenirlos en el futuro sistémicamente. SG1: determinar causas. SG2: abordar causas.

Tabla 8. Categoría 4. Alto rendimiento y madurez (HRM)

Sigla	Área de proceso	Nº Nivel	Descripción y metas (SG)
OPF	Enfoque en el proceso organizacional	Nivel 3	Planifica e implementa la mejora del proceso basándose en comprensión profunda de los procesos actuales. SG1: oportunidades. SG2: planificar e implementar. SG3: desplegar.
OPD	Definición del proceso organizacional	Nivel 3	Establece y mantiene los activos del proceso organizacional (OSP, plantillas, guías, métricas). SG1: establecer activos del proceso organizacional.
OT	Capacitación organizacional	Nivel 3	Desarrolla habilidades y conocimientos para que las personas desempeñen sus roles eficaz y eficientemente. SG1: capacidad de capacitación. SG2: proveer capacitación.
OPM	Gestión del desempeño organizacional	Nivel 5	Gestiona proactivamente el desempeño de la organización para alcanzar sus objetivos de negocio. SG1: gestionar desempeño. SG2: seleccionar mejoras.
OPP	Desempeño del proceso organizacional	Nivel 4	Establece comprensión cuantitativa del desempeño de los procesos estándar de la organización. SG1: líneas base y modelos de desempeño del proceso.

Metas genéricas y específicas: cada PA debe satisfacer sus metas específicas (SG) para cumplir con el nivel de madurez al que pertenece. Adicionalmente debe satisfacer las metas genéricas (GG) que demuestran la institucionalización del proceso en la organización.

- **Representación continua vs. Escalonada**

CMMI ofrece dos representaciones equivalentes para describir el mismo modelo. Ambas son válidas y conducen a los mismos resultados en términos de mejora de procesos; la diferencia está en el enfoque, la flexibilidad y la forma en que se comunican los resultados a la organización y al mercado.

A continuación, se explica cada una:

- **Representación escalonada (Staged representation)**

Organiza las áreas de proceso en cinco niveles de madurez predefinidos. Para avanzar de un nivel al siguiente, la organización debe satisfacer todas las metas del nivel actual. Proporciona una secuencia probada y predefinida para la mejora organizacional y tiene en cuenta:

- Evalúa: la organización completa con un único nivel de madurez (1–5).
- Resultado: un número único y fácilmente comunicable (ej: 'Somos Nivel 3 CMMI').
- Ruta: fija y predeterminada. No se puede omitir ningún nivel de madurez.

- Ventaja: facilita el benchmarking y la comparación entre organizaciones.
- Uso: estándar en licitaciones gubernamentales, contratos corporativos, certificaciones.
- Limitación: menor flexibilidad. No permite reflejar fortalezas específicas en áreas aisladas.

- **Representación continua (Continuous representation)**

Organiza las áreas de proceso en grupos funcionales (categorías) y permite a cada una ser evaluada con su propio nivel de capacidad del 0 al 3, independientemente de las demás. La organización define qué áreas son estratégicamente prioritarias. En ella se debe tener en cuenta:

- Evalúa: cada área de proceso individualmente con nivel de capacidad 0–3.
- Resultado: un perfil de capacidades detallado que muestra fortalezas y brechas por área.
- Ruta: personalizable. La organización define su target profile según objetivos de negocio.
- Ventaja: mayor flexibilidad y granularidad. Ideal para mejora focalizada.
- Uso: mejora interna estratégica, organizaciones con recursos limitados, start-ups.
- Ventaja adicional: permite identificar y capitalizar fortalezas diferenciadas por área.

De acuerdo a lo relacionado en la representación continua, cada nivel de capacidad representa:

- Nivel 0 — Incompleto. El proceso no se realiza o se realiza parcialmente. No todas las metas específicas se satisfacen.
- Nivel 1 — Realizado. El proceso se realiza y se logran las metas específicas. Sin rigor de gestión.
- Nivel 2 — Gestionado. El proceso se planifica, monitorea y controla. Los productos de trabajo son controlados.
- Nivel 3 — Definido. El proceso se adapta desde el proceso estándar de la organización y tiene documentación completa.

Para detallar de mejora manera estas dos representaciones, se relaciona la siguiente tabla comparativa:

Tabla 9. Comparación de representaciones CMMI

Dimensión	Escalonada (Staged)	Continua (Continuous)
Escala de medición	Niveles de Madurez 1–5	Niveles de Capacidad 0–3 por PA.
Unidad de evaluación	La organización completa.	Cada área de proceso individualmente.
Flexibilidad de ruta	Ruta fija y predeterminada.	Ruta personalizable (target profile).
Resultado de evaluación	Un número único (ej: Nivel 3).	Perfil de capacidades por área de proceso.
Uso principal	Benchmarking, contratos, certificación.	Mejora interna focalizada y estratégica.
Visibilidad de brechas	Baja (todo o nada por nivel).	Alta (fortalezas y debilidades por área).

Aplicación típica	Empresas que licitan contratos GOV.	Organizaciones con mejora continua interna.
Tiempo para evaluar	Mayor — evalúa todo simultáneamente.	Menor — se puede hacer por etapas.

Elección práctica: las organizaciones que compiten por contratos gubernamentales o corporativos, generalmente optan por la representación escalonada porque el mercado la reconoce y valora. Las organizaciones en proceso de mejora interna sin presión de certificación optan por la continua por su flexibilidad.

- **Implementación de CMMI en organizaciones**

Adoptar CMMI no es instalar un software ni crear documentación; es un proceso de transformación cultural y de procesos que típicamente toma entre 12 y 36 meses para alcanzar el nivel 3, dependiendo del tamaño de la organización, su cultura y su punto de partida. A continuación, la explicación de cada paso y su respectivo procedimiento:

- **Paso 1. Evaluación inicial (SCAMPI B/C) — Mes 1–2**

Realizar una evaluación diagnóstica para determinar el nivel de madurez actual y las brechas respecto al nivel objetivo.

- Contratar un lead appraiser certificado por el CMMI Institute.
- Seleccionar el nivel objetivo (típicamente nivel 2 o 3 como primera meta).
- Documentar el estado actual de los procesos vs. prácticas CMMI requeridas.

- Identificar y priorizar brechas por área de proceso.

- **Paso 2. Planificación de la mejora — Mes 2–3**

Diseñar un plan de mejora realista con responsables, recursos, cronograma e indicadores de éxito.

- Conformar el SEPG: Software Engineering Process Group (equipo multidisciplinario de implementación).
- Priorizar áreas de proceso según el nivel objetivo y las brechas identificadas.
- Estimar presupuesto: capacitación, consultoría, herramientas de soporte.
- Establecer indicadores de avance del plan de implementación.

- **Paso 3. Capacitación del equipo — Mes 3–6**

Entrenar a toda la organización en los conceptos CMMI y las prácticas requeridas. La resistencia al cambio es el mayor riesgo y se mitiga con capacitación que explique el porqué.

- Capacitación introductoria a CMMI para toda la organización (mandatorio).
- Entrenamiento específico por rol: PMs, QA, desarrolladores, arquitectos.
- Taller de interpretación y aplicación de prácticas específicas por PA.

- Certificación de personal clave como Practitioners o Instructors CMMI.

- **Paso 4. Definición e implementación de procesos — Mes 6–18**

Documentar y desplegar los procesos mejorados. Iniciar con un proyecto piloto para validar antes del despliegue general.

- Definir el Proceso Estándar de la Organización (OSP) con plantillas y guías.
- Implementar herramientas de apoyo: Jira (seguimiento), Confluence (documentación), SonarQube (calidad).
- Ejecutar proyecto piloto con los nuevos procesos documentados y supervisados.
- Refinar procesos con base en las lecciones aprendidas del proyecto piloto.

- **Paso 5. Medición y seguimiento continuo — Continuo - Meses 12–36 (y en adelante, de forma permanente)**

Recopilar datos, analizar métricas y realizar revisiones periódicas para verificar la adherencia a los procesos definidos.

- Implementar tablero de métricas de proceso (velocidad, defectos, cobertura, calidad).
- Realizar auditorías internas de proceso (PPQA) de forma trimestral.
- Ajustar procesos con base en datos aplicando el ciclo PHVA.

- Documentar lecciones aprendidas y actualizar los activos del proceso.

- **Paso 6. Evaluación formal SCAMPI A y certificación — Mes 24–36**

Realizar la evaluación oficial con un lead appraiser certificado para obtener la calificación de nivel, reconocida internacionalmente.

- Coordinar evaluación formal SCAMPI A con lead appraiser certificado.
- Preparar evidencias documentadas de cada práctica por cada área de proceso evaluada.
- Entrevistas al equipo por el equipo de evaluación (proceso de mínimo 2 días).
- Recibir el informe oficial y el registro en el portal cmmi.isaca.org.

Por su parte, los factores críticos de éxito son:

- Apoyo ejecutivo visible y sostenido: es el factor número uno de éxito o fracaso.
- Equipo SEPG dedicado con tiempo asignado formalmente a las actividades de mejora.
- Comenzar con un proyecto piloto: los resultados tangibles generan credibilidad interna.
- Involucrar al equipo técnico en la definición de procesos, no solo imponerlos desde arriba.

- No fabricar procesos de papel: CMMI evalúa institucionalización real, no documentación superficial.
- Visión de largo plazo: la presión por 'pasar la evaluación rápido' es el mayor destructor de valor.

La siguiente tabla, resume un poco lo que tiene que ver con las nivelaciones basadas en métricas:

Tabla 10. Beneficios medibles — ROI de la implementación

Métrica	Nivel 2	Nivel 3	Niveles 4–5	Fuente
Defectos en producción	↓ 15–25 %	↓ 35–50 %	↓ 60–80 %	CMMI Institute 2023
Tiempo de ciclo proyecto	Línea base	↓ 10–20 %	↓ 30–50 %	SEI Reports
Costo de retrabajo	↓ 10 %	↓ 30–40 %	↓ 60–70 %	Forrester Research
Cumplimiento de plazos	↑ 25–35 %	↑ 50–65 %	↑ 75–90 %	CMMI V2.0 Report
Satisfacción del cliente	↑ Moderada	↑ Alta	↑ Muy alta	CMMI Performance

CMMI en Colombia: cada vez más empresas colombianas de TI buscan certificarse en CMMI para participar en licitaciones del Estado o para exportar servicios de software. El tiempo promedio para alcanzar el nivel 3 en una empresa mediana (50–200 personas) es de 24 meses con un equipo SEPG dedicado y apoyo directivo sostenido.

4. Metodologías de desarrollo de software

Las metodologías de desarrollo de software son los marcos de trabajo que guían a los equipos desde la concepción de una idea hasta la entrega del producto final.

4.1. Metodologías tradicionales

Las metodologías tradicionales (o de "pesado") se caracterizan por ser disciplinadas, lineales y por requerir una documentación exhaustiva en cada etapa. Dentro de ellas, se encuentran los siguientes modelos:

A. Modelo en cascada (Waterfall)

Es el enfoque más antiguo y el pilar de la ingeniería de software tradicional. Se basa en una secuencia lógica de fases donde no se puede avanzar a la siguiente sin haber terminado la anterior.

- Funcionamiento: el proyecto fluye hacia abajo como una cascada, pasando por:
Requisitos
Diseño
Implementación
Verificación
Mantenimiento.

Dentro de sus aspectos positivos y negativos se encuentran:

- **Ventajas.** Es fácil de entender y gestionar debido a su rigidez. Funciona bien cuando los requisitos están perfectamente definidos desde el inicio.
- **Desventaja principal.** Es muy difícil y costoso retroceder. El cliente solo ve el resultado final al terminar todo el proceso, lo que genera riesgos si hubo malentendidos iniciales.

B. Modelo en V (V-Model)

Se considera una extensión del modelo en cascada, pero con un enfoque crítico en el aseguramiento de la calidad. Su nombre proviene de la forma en que se visualiza: el lado izquierdo representa la descomposición de requisitos y el derecho la integración y pruebas.

Relación directa: lo que hace especial a este modelo es que cada fase de desarrollo tiene una fase de prueba correspondiente. Por ejemplo, cuando se definen los requisitos, simultáneamente se planean las pruebas de aceptación.

Sus componentes son:

- **Rama descendente.** Requisitos del sistema, Diseño arquitectónico, Diseño de detalle.

Vértice: codificación.

- **Rama ascendente.** Pruebas de unidad (como la IEEE 1008), pruebas de integración, pruebas de sistema.

Ideal para: proyectos donde los fallos no son una opción (sistemas médicos o aeroespaciales), ya que la verificación es constante.

C. Modelo en espiral (Spiral)

Propuesto por Barry Boehm, este modelo rompe la linealidad de los anteriores introduciendo la gestión de riesgos como el eje central del desarrollo. Es un modelo evolutivo que combina la naturaleza iterativa con los aspectos controlados de la cascada.

Ciclos Iterativos: el proyecto se desarrolla en "espirales". Cada vuelta de la espiral pasa por cuatro cuadrantes:

- **Determinación de objetivos.** Identificar qué se quiere lograr en esa iteración.
- **Análisis de riesgos.** Evaluar qué puede salir mal y crear prototipos para mitigar fallas (esta es su mayor fortaleza).
- **Desarrollo y prueba.** Se construye la versión del software para ese ciclo.
- **Planificación.** Revisión del cliente y preparación para el siguiente ciclo.

Uso: es ideal para proyectos grandes, complejos y costosos donde el riesgo de fracaso debe ser monitoreado de cerca.

4.2. Metodologías ágiles

Estas metodologías ágiles surgen como una respuesta a la necesidad de flexibilidad, rapidez y entrega continua de valor en entornos donde los requisitos cambian constantemente. En dicha metodología se encuentra:

A. Manifiesto ágil (Agile anifesto)

Publicado en 2001, es el cimiento de todas las metodologías ágiles. Se basa en cuatro valores fundamentales que priorizan a las personas y los resultados sobre los procesos rígidos:

- Individuos e interacciones sobre procesos y herramientas.
- Software funcionando sobre documentación extensiva.
- Colaboración con el cliente sobre negociación contractual.
- Respuesta ante el cambio sobre seguir un plan.

La siguiente imagen explica este manifiesto:

Figura 1. Manifiesto ágil



B. Scrum

Es el marco de trabajo ágil más popular. Se basa en el desarrollo iterativo e incremental, mediante ciclos de tiempo fijos llamados sprints (generalmente de 2 a 4 semanas) e incluye:

- Roles clave (Product Owner): representa al cliente y prioriza el trabajo.
 - Scrum master: facilita el proceso y elimina obstáculos.
 - Equipo de desarrollo: profesionales que construyen el software.
- Artefactos: el product backlog (lista de tareas pendientes) y el sprint backlog (tareas para el ciclo actual).
- Rituales: reuniones diarias (Daily Stand-up), revisión del sprint y retrospectiva.

C. Kanban

Es un método visual para gestionar el trabajo a medida que avanza por un proceso. A diferencia de Scrum, no tiene roles fijos ni ciclos de tiempo obligatorios; se enfoca en el flujo continuo, aplicando:

- Tablero Kanban: se divide en columnas (ej. Por hacer, En curso, Hecho).
- WIP (Work In Progress): limita la cantidad de tareas que pueden estar en una columna al mismo tiempo para evitar cuellos de botella.
- Objetivo: visualizar el trabajo, reducir el tiempo de entrega y mejorar la eficiencia del equipo de manera constante.

D. DevOps

Más que una metodología, es una filosofía y cultura que busca unificar el desarrollo de software (Dev) con las operaciones de TI (Ops). Su meta es acortar el ciclo de vida del desarrollo y proporcionar una entrega continua de alta calidad.

Igualmente incluye:

- Ciclo infinito: representa la integración entre planear, codificar, construir, probar, liberar, desplegar, operar y monitorear.
- Automatización: es fundamental para DevOps. Incluye conceptos como:
 - CI (integración continua): subir código y probarlo automáticamente varias veces al día.
 - CD (entrega continua): desplegar automáticamente las mejoras para que el usuario las reciba rápido.

A continuación, se presenta una tabla comparativa que permite contrastar las principales diferencias entre las metodologías tradicionales y las metodologías ágiles:

Tabla 11. Características de las metodologías tradicionales y las metodologías ágiles

Característica	Metodologías tradicionales (Cascada, V, Espiral)	Metodologías ágiles (Scrum, Kanban, DevOps)
Enfoque principal	Procesos y documentación exhaustiva.	Individuos e interacciones.
Requisitos	Se definen totalmente al inicio (estáticos).	Cambiantes y evolucionan con el proyecto.
Entrega de valor	Al final del ciclo de vida del proyecto.	Entregas pequeñas y funcionales constantes.
Estructura	Lineal o secuencial por fases.	Iterativa e incremental (sprints).

Característica	Metodologías tradicionales (Cascada, V, Espiral)	Metodologías ágiles (Scrum, Kanban, DevOps)
Cambios	Difíciles de implementar y costosos.	Bienvenidos, incluso en etapas tardías.
Relación con cliente	Contractual; el cliente interviene al inicio y final.	Colaborativa; el cliente es parte del equipo.
Equipo	Jerárquico y con roles muy especializados.	Autogestionado y multidisciplinario.
Éxito medido por...	Cumplimiento del plan y documentación.	Software funcionando que satisface al usuario.

4.3. Calidad en entornos ágiles vs tradicionales

La gestión de la calidad es uno de los puntos donde más chocan ambas filosofías. Mientras que en los entornos tradicionales la calidad es una etapa de control (una barrera al final), en los entornos ágiles la calidad es un hábito continuo (integrada en cada paso).

- **La calidad en entornos tradicionales**

En modelos como la cascada o el modelo en V, la calidad se percibe como una fase de inspección técnica e incluye:

- **Momento de ejecución.** Se realiza principalmente después de la fase de construcción (codificación). Existe una separación clara entre los desarrolladores y el equipo de QA (Quality Assurance).

- **Enfoque.** Se basa en el cumplimiento estricto de los requisitos documentados al inicio. Si el software hace lo que dice el documento, tiene "calidad", aunque el cliente ya no necesite esa funcionalidad.
- **Documentación.** Se generan planes de prueba masivos, reportes de incidentes y certificados de calidad formales.
- **Riesgo.** El mayor peligro es encontrar un error arquitectónico al final del proceso, lo que implica retroceder meses de trabajo y elevar los costos exponencialmente.

- **La calidad en entornos ágiles**

En Scrum, Kanban o DevOps, la calidad no es una fase, es una responsabilidad compartida de todo el equipo durante todo el ciclo de vida y por ello, se presentan las siguientes acciones:

- **Calidad construida (Built-in Quality).** En lugar de esperar a que algo falle para arreglarlo, se aplican prácticas para no introducir errores. Se prueba el código cada vez que se escribe (siguiendo estándares como IEEE 1008).
- **Enfoque.** Se centra en el valor para el usuario. La calidad se define por la satisfacción del cliente y la capacidad de respuesta ante sus cambios.
- **Pruebas continuas.** Se utiliza la automatización (CI/CD) para ejecutar pruebas cada pocas horas. Si algo se rompe, el equipo se entera de inmediato.
- **Definición de hecho (Definition of done).** Un requisito no se considera terminado solo porque el código esté escrito, debe haber

pasado pruebas unitarias, de integración y revisión por pares para considerarse de "calidad".

Para clarificar un poco este tema de calidad, se presenta la siguiente tabla:

Tabla 12. Comparativo de calidad

Aspecto	Calidad tradicional	Calidad ágil
Responsabilidad	Del equipo de QA / Testers.	De todo el equipo (incluyendo programadores).
Cuándo ocurre	Al final (fase de pruebas).	Constantemente (en cada sprint o tarea).
Objetivo	Cero defectos vs especificaciones.	Valor para el negocio y mejora continua.
Costo de error	Muy alto (se detecta tarde).	Bajo (se detecta en horas o días).
Automatización	Opcional / Secundaria.	Crítica y fundamental (DevOps).

5. Disciplinas de calidad de software

Son el conjunto de áreas de conocimiento, prácticas y procesos estandarizados que aseguran que un producto de software no solo funcione, sino que sea confiable, eficiente y fácil de mantener.

5.1. Personal Software Process (PSP)

Es un modelo de desarrollo diseñado para ayudar a los ingenieros de software a organizar su trabajo individual, mejorar su productividad y garantizar la calidad de los productos que entregan. Fue creado por Watts Humphrey con la idea de llevar la disciplina del modelo organizacional CMMI al nivel del programador individual.

- **Principios del PSP**

El PSP se basa en la premisa de que la calidad de un sistema de software depende de la calidad del trabajo de cada uno de los ingenieros que lo integran. Sus principios fundamentales son:

- **Responsabilidad personal.** Cada desarrollador es responsable de la calidad de su código.
- **Mejora continua.** Para mejorar, un profesional debe planificar su trabajo, medir su desempeño y actuar según los resultados.
- **Análisis de datos.** El uso de datos históricos personales es la única forma confiable de estimar tiempos y predecir errores en futuros proyectos.
- **Enfoque en la calidad.** Es mucho más eficiente encontrar y corregir errores en las etapas iniciales (diseño y codificación) que durante las pruebas finales.

- **Niveles de PSP**

El proceso se introduce de forma gradual a través de diferentes niveles, permitiendo que el desarrollador adopte la disciplina paso a paso, la cual se explica a continuación:

Tabla 13. Niveles PSP

Nivel	Nombre	Enfoque principal
PSP 0	Proceso actual	Establece una línea base. Se registran los tiempos y defectos del proceso que el programador ya usa.
PSP 0.1	Estándares	Introduce estándares de codificación, medición de tamaño del software y un diario de procesos.
PSP 1	Planificación	Se enfoca en la estimación de tamaño y tiempo basada en datos históricos.
PSP 1.1	Calendario	Incluye la planificación de cronogramas y el seguimiento del progreso del proyecto.
PSP 2	Calidad y diseño	Introduce revisiones de diseño y código para detectar errores antes de compilar.
PSP 2.1	Plantillas	Uso de plantillas de diseño y análisis de verificación para asegurar robustez.

- **Medición personal**

La medición es el núcleo del PSP, ya que sin datos no existe mejora objetiva, por ello los desarrolladores utilizan formularios y hojas de registro para capturar estas tres métricas esenciales:

- **Tiempo.** Registro minucioso de los minutos dedicados a cada fase (planificación, diseño, codificación, revisión, compilación, pruebas y post-mortem). Se descuentan interrupciones para obtener el "tiempo neto".
- **Defectos.** Se registra cada error encontrado, la fase en la que se introdujo, la fase en la que se detectó y el tiempo que tomó corregirlo. Esto ayuda a calcular el rendimiento (porcentaje de defectos eliminados antes de las pruebas).
- **Tamaño.** Generalmente medido en líneas de código (LOC) o puntos de función. Permite relacionar cuánto tiempo toma desarrollar una cantidad específica de software.

5.2. Team Software Process (TSP)

Es el siguiente paso lógico tras el PSP. Mientras que el PSP se enfoca en la disciplina individual, el TSP proporciona el marco de trabajo para que equipos de ingenieros produzcan software de alta calidad, a tiempo y dentro del presupuesto.

- **Estructura del equipo**

El TSP organiza a los desarrolladores en equipos auto-dirigidos. No se trata de un grupo con un jefe que da órdenes, sino de un equipo que:

- **Establece sus propios objetivos.** Alineados con las metas del negocio.
- **Planifica su propio trabajo.** Los miembros deciden cuánto trabajo pueden realizar en un ciclo.

- **Monitorea su propio progreso.** El equipo es responsable de su éxito o fracaso.
- **Mantiene la calidad.** El equipo define sus propios estándares de calidad basándose en datos.

- **Roles en el TSP**

Para que un equipo auto-dirigido funcione sin un gerente tradicional, el TSP asigna roles específicos a sus miembros. Un solo integrante puede tener más de un rol si el equipo es pequeño. Dentro de los roles se incluyen:

- **Líder del equipo (Team leader).** Dirige al equipo, asegura que todos participen y reporta el progreso a la gerencia.
- **Gestor de desarrollo (Development manager).** Lidera la creación del diseño y la escritura del código.
- **Gestor de planificación (Planning manager).** Consolida los planes individuales y rastrea el avance del cronograma del equipo.
- **Gestor de calidad (Quality manager).** Administra el plan de calidad y asegura que se realicen las revisiones de código y diseño.
- **Gestor de soporte/herramientas (Support manager).** Gestiona la configuración del software y las herramientas necesarias para el desarrollo.

- **Métricas de equipo**

El éxito del TSP se basa en la medición. Al combinar los datos de los PSP individuales, el equipo obtiene una visión clara de su desempeño colectivo mediante:

- **Esfuerzo del equipo.** Suma de las horas netas de trabajo de todos los miembros contra lo planificado.
- **Densidad de defectos.** Número de errores encontrados por cada mil líneas de código (KLOC).
- **Calidad de la revisión.** Porcentaje de defectos eliminados en las fases de revisión antes de llegar a las pruebas del sistema.
- **Valor ganado (Earned value).** Una métrica que indica cuánto del proyecto se ha completado realmente en términos de valor, no solo de tiempo transcurrido.

- **Integración con CMMI**

El TSP actúa como el "puente operacional" para alcanzar los niveles de madurez del CMMI (Capability Maturity Model Integration). Esta integración incluye:

- **Implementación práctica.** Mientras CMMI define qué debe hacer una organización (los objetivos), el TSP define cómo hacerlo a nivel de proyecto.
- **Niveles de madurez.** Una organización que utiliza TSP de manera consistente puede alcanzar rápidamente el nivel 3 (Definido) y el nivel 5 (Optimizado) de CMMI, ya que el TSP genera automáticamente los datos y evidencias necesarios para las auditorías.

- **Institucionalización.** TSP asegura que los procesos de calidad de CMMI no se queden en documentos, sino que se vivan diariamente en el desarrollo del software.

6. Administración del proceso personal de construcción de software

Es una disciplina técnica diseñada para que el desarrollador gestione de forma individual la calidad, el tiempo y la productividad de su propio trabajo. Se basa en el principio de que, para mejorar un proceso, primero es necesario medirlo; por ello, utiliza el registro minucioso de datos históricos sobre defectos encontrados y tiempos dedicados a cada fase del desarrollo.

6.1. Fundamentos y principios

El PSP se basa en la idea de que la calidad de un sistema de software está determinada por el eslabón más pequeño: el trabajo individual del programador.

- **Disciplina personal**

La disciplina es el núcleo del proceso. No se trata de seguir reglas impuestas externamente, sino de que el profesional adopte un compromiso propio para:

- Seguir un proceso definido de manera consistente.
- Registrar datos de forma honesta y precisa (tiempos, interrupciones y errores).
- Analizar su propio desempeño para identificar debilidades.
- Mantener el rigor técnico incluso bajo la presión de los plazos de entrega.

- **Planeación basada en datos históricos**

Uno de los principios más disruptivos del PSP es que no se permite adivinar. Las estimaciones de tiempo y esfuerzo se realizan mediante métodos estadísticos basados en el pasado del propio desarrollador. Se aplica:

- **Estimación de tamaño.** Se utiliza la experiencia previa para predecir cuántas líneas de código o funciones tendrá el nuevo programa.
- **Predicción de tiempos.** Si históricamente un docente de tecnología tarda 10 minutos por cada 100 líneas de código, ese dato se usa para planificar la siguiente guía de trabajo.
- **Precisión.** Con el tiempo, el error de estimación disminuye, permitiendo compromisos de entrega reales y alcanzables.

6.2. Niveles del PSP (PSP0 a PSP3)

El aprendizaje del PSP es incremental. El profesional sube de nivel a medida que domina nuevas técnicas de medición y control. Por consiguiente, se presentan los distintos niveles del PSP, desde el proceso base hasta el enfoque cíclico, los cuales permiten al desarrollador incorporar progresivamente prácticas de medición, planeación y calidad a medida que avanza en su dominio del método:

- **PSP0 y PSP0.1: el proceso base**
 - PSP0: el desarrollador utiliza su método actual, pero comienza a registrar el tiempo dedicado y los defectos encontrados. Introduce el uso de formularios simples.

- PSP0.1: se añaden estándares de codificación y se introduce la medición del tamaño del software (LOC - líneas de código).
- **PSP1 y PSP1.1: el proceso de planeación**
 - PSP1: introduce el método PROBE (Proxy Based Estimation) para estimar el tamaño y el tiempo basándose en datos históricos.
 - PSP1.1: se añade la planeación de tareas y el cronograma del proyecto (calendario).
- **PSP2 y PSP2.1: el proceso de calidad**
 - PSP2: es el nivel más crítico. Se introducen las revisiones de diseño y de código. El objetivo es encontrar errores antes de compilar y se aprende que revisar es más eficiente que probar.
 - PSP2.1: introduce el diseño jerárquico y plantillas de diseño para asegurar que el código sea robusto desde su concepción.
- **PSP3: el proceso cíclico**
 - PSP3: está diseñado para proyectos más grandes que no caben en una sola iteración de PSP2. Permite subdividir el proyecto en módulos pequeños y aplicar los niveles anteriores de forma cíclica. Es la base para trabajar después en equipos con TSP.

6.3. Instrumentos de control

A continuación, se describen los principales instrumentos de control del PSP, los cuales proporcionan estructura, disciplina y consistencia al trabajo del desarrollador, permitiendo planificar, medir y mejorar su desempeño de manera sistemática:

- **Scripts (Guiones de proceso)**

Los scripts son guías paso a paso que describen exactamente qué actividades realizar en cada fase del desarrollo. Su función es garantizar que el proceso sea repetible y que no se salte ninguna etapa crítica.

- Contenido: definen las condiciones de entrada (qué necesito para empezar), los pasos a seguir (ej. diseño, codificación, revisión) y las condiciones de salida (qué debo entregar).
- Utilidad: ayudan al programador a mantener el enfoque y la disciplina, especialmente en fases que suelen omitirse, como la revisión de diseño o el análisis de post-mortem.

- **Formas (Plantillas de registro)**

Las formas son los documentos o interfaces donde se recolectan los datos crudos durante el trabajo. Son el "diario" del ingeniero de software.

- Tipos comunes: resumen del proyecto (project plan summary), formas de diseño y registros de análisis.
- Importancia: proporcionan una estructura uniforme para capturar datos. Al usar siempre la misma forma, el desarrollador puede comparar proyectos realizados en enero con proyectos de junio de manera objetiva.

- **Estándares personales**

Para que las mediciones sean válidas, el desarrollador debe establecer sus propias reglas de juego. Sin estándares, los datos son inconsistentes.

- Estándar de codificación: define cómo se debe escribir el código (nombres de variables, sangría, comentarios). Esto facilita encontrar errores durante las revisiones.
- Estándar de defectos: una lista que clasifica los errores (ej. Tipo 10: sintaxis, Tipo 20: lógica, Tipo 30: interfaz). Esto permite saber en qué área el desarrollador suele fallar más.
- Estándar de tamaño: define cómo se medirá el producto (generalmente líneas de código o funciones) para que la estimación siempre se base en la misma unidad.

- **Registro de tiempos y defectos**

Este instrumento permite al desarrollador capturar de manera sistemática el tiempo invertido en cada fase del proceso y los defectos detectados a lo largo del desarrollo.

- Contenido: incluye la fase del proceso, fecha, tiempo de inicio y fin, interrupciones, tipo de defecto, fase de detección y fase de inyección del error.
- Utilidad: facilita el análisis de productividad y calidad, permite identificar en qué fases se consume más tiempo y dónde se originan la mayoría de los defectos, apoyando la toma de decisiones para optimizar el proceso personal.

6.4. Método PROBE

Es el componente del PSP que permite a los desarrolladores realizar estimaciones precisas de tamaño y tiempo. En lugar de "adivinar", el ingeniero utiliza su propia historia y herramientas estadísticas para predecir el futuro del proyecto.

- **Estimación basada en objetos (Proxies)**

La dificultad de estimar software radica en que es intangible. PROBE soluciona esto usando Proxies (objetos representativos).

¿Qué es un proxy?: es una entidad física o conceptual que se puede contar y que tiene una relación fuerte con el esfuerzo final. El proxy más común son los objetos o funciones del programa.

El proceso: el desarrollador identifica los objetos que necesita construir (ej. una función de búsqueda, una base de datos) y los clasifica por tamaño (muy pequeño, pequeño, medio, grande, muy grande) basándose en su tabla de datos históricos.

Cálculo de tamaño: al sumar el tamaño estimado de todos estos "objetos", se obtiene una estimación del tamaño total del sistema antes de escribir una sola línea de código.

- **Análisis estadístico**

PROBE no confía en la intuición, sino en los datos. El análisis estadístico se utiliza para validar si las estimaciones pasadas del desarrollador son lo suficientemente buenas como para predecir el futuro.

Calidad de los datos: se evalúa la correlación entre lo que el desarrollador estimó en el pasado y lo que realmente construyó.

Intervalos de confianza: el método calcula un rango de probabilidad (por ejemplo, con un 70 % o 90 % de confianza). Esto permite al ingeniero decir: "Este proyecto me tomará 40 horas, con un margen de error de +/- 4 horas".

Significancia: se utilizan pruebas estadísticas para asegurar que la relación entre el tamaño del objeto y el tiempo de desarrollo no sea una coincidencia, sino un patrón real.

- **Regresión lineal simple**

Es la herramienta matemática central del método PROBE. Se utiliza para convertir el tamaño estimado (proxy) en tiempo real de desarrollo.

La Fórmula: se utiliza la ecuación de la recta:

$$y = \beta_0 + \beta_1 x$$

y: tiempo total estimado.

x: tamaño del proxy (líneas de código u objetos).

β_0 (Pendiente): indica cuánto tiempo aumenta por cada unidad de tamaño.

β_1 (Intersección): representa el tiempo fijo o "costo de arranque" de cualquier proyecto.

Aplicación: si los datos históricos del desarrollador muestran una línea recta consistente al graficar "Tamaño vs. Tiempo", la regresión lineal permite proyectar con gran exactitud cuánto tardará un nuevo proyecto simplemente ingresando el tamaño estimado de los objetos.

6.5. Estadísticas aplicadas

Las estadísticas aplicadas no son simples números, sino la base científica que permite a un desarrollador entender su capacidad real. Sin estos datos, un ingeniero está "adivinando"; con ellos, está "gestionando".

- **Tamaño (Size)**

El tamaño es la variable independiente básica en la estadística de software. Es el dato que indica qué tan grande o complejo es el trabajo realizado e incluye los siguientes aspectos:

- **Medición.** Aunque existen varias unidades (como puntos de función), en PSP se utilizan comúnmente las líneas de código (LOC). Se miden las líneas nuevas, las cambiadas y las reutilizadas.
- **Importancia estadística.** El tamaño es el principal predictor del esfuerzo. Si se conoce el tamaño probable de un módulo, se puede usar para calcular el tiempo necesario, mediante la regresión lineal.
- **Normalización.** Permite comparar proyectos de distinta magnitud. No es lo mismo encontrar 5 errores en un programa de 10 líneas que en uno de 1,000.

- **Productividad (Productivity)**

La productividad mide la eficiencia del desarrollador. En el entorno personal, ayuda a establecer expectativas realistas de entrega. Tiene presente las siguientes acciones:

- **Cálculo.** Se define generalmente como la relación entre el tamaño y el tiempo neto dedicado:

$$\text{Productividad} = (\text{Unidades de tamaño (LOC)}) / (\text{Horas de Esfuerzo})$$

- **Uso práctico.** Conocer su productividad media (ej. 25 LOC por hora), permite planificar jornadas de trabajo. Si una guía de tecnología requiere programar 200 líneas, el usuario sabe estadísticamente que necesitará unas 8 horas de trabajo neto.
- **Variabilidad.** El PSP enseña que la productividad no es constante; varía según el lenguaje de programación, la dificultad del algoritmo y la experiencia previa.

- **Densidad de defectos (Defect density)**

Es la métrica de oro para medir la calidad del software. Indica cuántos errores se introducen por unidad de tamaño y se aplica de la siguiente manera:

- **Cálculo.** Se obtiene dividiendo el total de defectos encontrados entre el tamaño del software, usualmente expresado en defectos por cada mil líneas de código (KLOC):

$$\text{Densidad de defectos} = (\text{Total de Defectos}) / \text{KLOC}$$

- **Interpretación.** Una densidad alta sugiere que el proceso de codificación es descuidado o que no se están realizando suficientes revisiones previas.
- Una densidad baja (después de las pruebas) indica un producto robusto y confiable.

- **Benchmarking:** permite al desarrollador establecer metas de mejora. Un objetivo común en PSP es reducir la densidad de defectos en la fase de "Pruebas" aumentando la detección en la fase de "Revisión de Código".

6.6. Herramientas informáticas de apoyo

A continuación, se presenta una tabla que resume las principales herramientas informáticas que apoyan la aplicación del PSP, facilitando la medición, el control y el análisis del desempeño del desarrollador a lo largo del proceso:

Tabla 14. Herramientas informáticas de apoyo para la implementación del PSP

Categoría	Herramientas recomendadas	Función principal en el proceso
Software especializado en PSP	Process Dashboard, PSP Student Workbench	Permiten el registro automático de tiempos, gestión de scripts de proceso y generación de planes de proyecto basados en PROBE.
Control de tiempos (Time logs)	Toggl Track, Clockify, Pomotodo	Ideales para medir el tiempo neto de desarrollo, permitiendo pausar durante interrupciones para obtener métricas reales de esfuerzo.
Análisis estadístico y PROBE	Microsoft Excel, Google Sheets	Imprescindibles para realizar la regresión lineal, calcular intervalos de confianza y mantener la base de datos históricos personales.
Gestión de defectos (Defect logs)	Jira, GitHub Issues, Trello	Facilitan el registro de cada error, su categoría (sintaxis, lógica, etc.) y el tiempo que tomó su corrección.
Conteo de tamaño (LOC)	CLOC (Count Lines of Code), LineCounter	Automatizan el conteo de líneas de código (nuevas, borradas y modificadas) para alimentar la métrica de tamaño.

Categoría	Herramientas recomendadas	Función principal en el proceso
Estándares de codificación	SonarQube, ESLint, Prettier	Ayudan a mantener la disciplina personal, mediante la revisión automática de reglas de estilo y detección temprana de errores comunes.

7. Documentación en procesos de calidad

Actúa como un registro objetivo que transforma las actividades técnicas en evidencias auditables y repetibles dentro de un marco de ingeniería. Su función es estandarizar los criterios de aceptación, mediante planes y reportes que aseguran que cada fase del desarrollo, desde la definición de requisitos hasta las pruebas finales, cumpla con las normas establecidas.

7.1. Estándares de documentación

Son el lenguaje común que permite que la calidad sea medible y auditable. En ingeniería de software, estos documentos no son simples trámites, sino herramientas de gestión que aseguran que el equipo sepa qué debe cumplir y cómo se verificará dicho cumplimiento.

- **Plan de aseguramiento de calidad (SQAP)**

El software Quality Assurance Plan es el documento maestro que define el "qué" y el "quién". Su objetivo es establecer el marco de trabajo para prevenir defectos antes de que ocurran. Sus acciones son las siguientes:

- **Alcance.** Define qué partes del proyecto serán supervisadas (código, manuales de usuario, requisitos).
- **Roles.** Determina quién es el responsable de auditar los procesos y quién de aprobar las entregas.
- **Normativas.** Cita los estándares internacionales que se seguirán, como ISO/IEC 25010 o la IEEE 1012 que vimos anteriormente.

- **Revisiones y auditorías.** Establece un calendario de inspecciones para asegurar que el equipo no se desvíe del proceso definido.

- **Plan de pruebas (Software test plan)**

El plan de pruebas es el documento que define cómo, cuándo y con qué criterios se verificará que el software cumple los requisitos establecidos. Sus principales componentes son los siguientes:

- **Alcance de las pruebas.** Especifica qué funcionalidades o requisitos serán evaluados y cuáles quedan fuera del ciclo de pruebas.
- **Estrategia de prueba.** Define los tipos de pruebas a ejecutar y el enfoque general para su realización.
- **Criterios de aceptación.** Establece las condiciones que deben cumplirse para considerar una prueba como aprobada y dar por finalizado el proceso.

- **Procedimientos (Standard operating procedures)**

Los procedimientos son el nivel más operativo. Son las "instrucciones de uso" detalladas sobre cómo ejecutar las tareas de calidad.

Naturaleza: son guías paso a paso (similares a los scripts de PSP) que aseguran que cualquier miembro del equipo pueda realizar una tarea de la misma manera.

Los ejemplos comunes son:

- **Procedimiento de revisión de código.** Pasos para subir una solicitud de cambio y cómo debe revisarla un compañero.

- **Procedimiento de reporte de incidencias.** Cómo llenar un ticket de error para que sea claro y útil para el programador.
- **Procedimiento de gestión de versiones.** Cómo usar herramientas como Git para que el código no se pierda.

7.2. Formatos y plantillas

Son las herramientas que materializan la calidad. Su objetivo es asegurar que la información crítica no dependa de la memoria de los desarrolladores, sino que quede registrada de forma estructurada para su análisis posterior.

- **Reportes de defectos (Bug reports)**

Un reporte de defectos es el documento que comunica un fallo a los desarrolladores. Para que sea efectivo, debe ser claro, reproducible y objetivo. Sus componentes esenciales son:

- **ID único.** Para seguimiento.
- **Título descriptivo.** Resumen del error.
- **Pasos para reproducir.** Instrucciones exactas para que el programador vea el fallo.
- **Resultado esperado vs. Resultado real.** Lo que debía pasar frente a lo que pasó.
- **Severidad y prioridad.** Qué tan grave es y qué tan rápido debe arreglarse.

Importante: un buen formato de reporte reduce el "rebote" de información entre el equipo de pruebas y el de desarrollo, acelerando las correcciones.

- **Registro de métricas (Metrics log)**

El registro de métricas es el almacén de datos cuantitativos del proyecto. Es el insumo principal para el análisis estadístico que se indicó en el PSP. Los datos a recolectar son:

- Tiempo neto de desarrollo en las horas invertidas por fase.
- Tamaño del software en las líneas de código o puntos de función.
- Defectos encontrados en la clasificación por tipo y fase donde se detectaron.

Propósito: estas plantillas permiten calcular la productividad y la densidad de defectos. Sin un formato estandarizado, es imposible comparar el desempeño entre diferentes proyectos o periodos académicos.

- **Trazabilidad (Traceability)**

La trazabilidad es la capacidad de seguir el rastro de un elemento de software a lo largo de todo su ciclo de vida. Se gestiona usualmente mediante una Matriz de Trazabilidad. En la conexión de elementos se debe asegurar que cada requisito del cliente tenga:

- Un componente de diseño que lo soporte.
- Un bloque de código que lo implemente.
- Un caso de prueba que lo verifique.

Utilidad: si un requisito cambia, la trazabilidad permite saber inmediatamente qué partes del código y qué pruebas deben actualizarse. También garantiza que no se haya construido nada "extra" que el cliente no pidió.

7.3. Gestión documental

Es el conjunto de prácticas diseñadas para organizar, almacenar y rastrear todos los artefactos de un proyecto (código, manuales, planes de calidad). Su objetivo es garantizar que la información sea íntegra, esté actualizada y sea accesible para todo el equipo.

A continuación, se abordan los principales componentes de la gestión documental en proyectos de software, los cuales permiten asegurar el control, la trazabilidad y la confiabilidad de la información a lo largo del ciclo de vida del desarrollo:

- **Control de versiones**

Es el mecanismo que permite registrar los cambios realizados sobre uno o varios archivos a lo largo del tiempo. A través de este sistema, múltiples desarrolladores pueden trabajar de forma simultánea sin sobrescribir el trabajo de otros, manteniendo un flujo de trabajo controlado. Cada modificación queda documentada mediante commits, los cuales registran quién realizó el cambio, cuándo se efectuó y por qué, lo que facilita la trazabilidad. Asimismo, la ramificación posibilita la creación de líneas de trabajo independientes, como la corrección de errores urgentes o el desarrollo de nuevas funcionalidades, que posteriormente pueden integrarse de manera controlada al proyecto principal.

- **Repositorios**

Son los espacios de almacenamiento digital donde se conservan los archivos del proyecto, junto con su historial completo de versiones. Estos pueden ser locales, residiendo en el equipo del desarrollador, o remotos, alojados en servidores

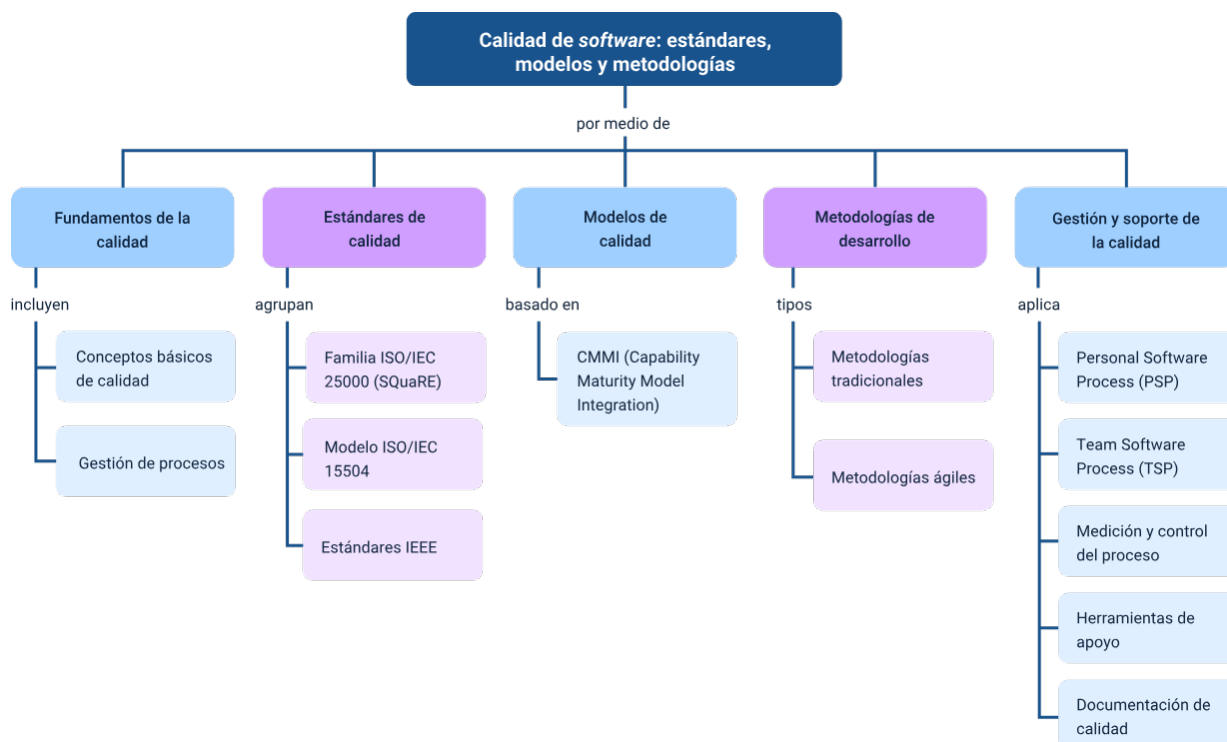
compartidos o en la nube para facilitar el trabajo colaborativo. Su función principal es actuar como la única fuente de verdad del proyecto, ya que todo artefacto relevante debe estar versionado y almacenado en el repositorio para ser considerado válido dentro del proceso de calidad.

- **Auditoría documental**

Consiste en la revisión sistemática de la documentación del proyecto para verificar que cumpla con los estándares establecidos y refleje fielmente el estado real del software. Este proceso permite asegurar que no existan brechas entre lo planificado y lo ejecutado, comprobando la existencia de planes de prueba, registros de métricas y manuales actualizados. Además, es un elemento clave para el cumplimiento normativo en evaluaciones o certificaciones, ya que confirma que los cambios realizados cuentan con respaldo documental, siguen el flujo de aprobación definido y no han sido alterados de forma no autorizada.

Síntesis

Este componente formativo se centra en el aseguramiento de la calidad del software mediante la integración de estándares internacionales, modelos de madurez y metodologías de desarrollo. El documento aborda desde conceptos fundamentales como la distinción entre calidad de proceso y de producto, hasta la aplicación de marcos normativos como ISO/IEC 25000 (SQuaRE), CMMI y diversos estándares IEEE. Además, detalla disciplinas técnicas para la mejora de la productividad individual y colectiva, tales como el PSP (Personal Software Process) y el TSP (Team Software Process). Finalmente, el contenido enfatiza la importancia de la mejora continua, basada en el ciclo PHVA y la gestión mediante indicadores y métricas para garantizar productos tecnológicos confiables que satisfagan las necesidades del usuario y de las organizaciones.



Glosario

Análisis estático: evaluación del código fuente (sin ejecutarlo) para detectar problemas estructurales, vulnerabilidades o incumplimiento de estándares.

Densidad de defectos: indicador que mide la cantidad de fallos encontrados en relación al tamaño del software (por ejemplo, defectos por cada mil líneas de código).

Deuda técnica: riesgo acumulado por decisiones de desarrollo rápidas o de baja calidad que requieren refactorización futura para evitar fallos.

Mantenibilidad: característica de calidad que evalúa la facilidad con la que el software puede ser modificado, incluyendo modularidad y testabilidad.

SQuaRE (ISO 25000): familia de estándares internacionales para la especificación y evaluación de los requisitos de calidad del producto software.

Trazabilidad: capacidad técnica de conectar un requisito con su código, pruebas ejecutadas y defectos asociados a lo largo del ciclo de vida.

UAT (User Acceptance Testing): pruebas de validación final realizadas por los usuarios para confirmar que el sistema cumple con sus expectativas de negocio.

Validación: proceso que asegura que el producto construido es el correcto y satisface las necesidades reales del usuario final.

Verificación: proceso que evalúa si el producto se está construyendo correctamente de acuerdo con las especificaciones técnicas.

Referencias bibliográficas

Humphrey, W. S. (1995). A discipline for software engineering. Addison-Wesley.

Humphrey, W. S. (2000). Introduction to the Team Software Process. Addison-Wesley.

Institute of Electrical and Electronics Engineers (IEEE). (2008). IEEE Std 730-2002: IEEE standard for software quality assurance plans. IEEE.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico – Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Solanlly Sánchez Melo	Experta temática	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Juan Daniel Polanco Muñoz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Veimar Celis Meléndez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Francisco José Vásquez Suárez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Ernesto Navarro Jaimes	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Animador y productor audiovisual
María Fernanda Pineda Mora	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima

Nombre	Cargo	Centro de Formación y Regional
Javier Mauricio Oviedo	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima